Um eine *jar*-Datei ausführbar zu machen, muss festgelegt werden, welche die Startklasse ist. (Sie erinnern sich: Die Methode zum Starten heißt immer **main**, aber wir müssen angeben, in welcher Startklasse diese Methode liegt.) Dazu wird in einer *jar*-Datei eine Textdatei abgelegt (die so genannte *Manifest-Datei*), die diese Information enthält. BlueJ bietet hierbei Unterstützung an.

Um eine ausführbare *jar*-Datei in BlueJ zu erstellen, können Sie im Menü PROJEKT den Eintrag ALS JAR-ARCHIV SPEICHERN aufrufen und im folgenden Dialog die Klasse auswählen, die die **main**-Methode enthält. (Sie müssen die **main**-Methode wie oben beschrieben implementieren.)

Für weitere Fragen zu dieser Operation können Sie das BlueJ-Tutorial lesen, das Sie im Menü HILFE über den Eintrag BLUEJ-TUTORIAL oder auf den BlueJ-Webseiten finden. Sobald die ausführbare *jar*-Datei erzeugt wurde, kann sie mit einem Doppelklick gestartet werden. Dazu muss auf dem Rechner, der die *jar*-Datei ausführen soll, ein JDK (ein Java Development Kit) oder ein JRE (Java Runtime Environment) installiert und mit *jar*-Dateien assoziiert sein.

E.4 Entwickeln ohne BlueJ

Wenn Sie Ihre Programme nicht nur ohne BlueJ ausführen, sondern auch ohne BlueJ entwickeln wollen, dann müssen Sie Ihre Klassen editieren und übersetzen. Der Quelltext einer Klasse wird in einer Datei mit der Endung *.java* gespeichert. Beispielsweise ist die Klasse **Spiel** in einer Datei mit dem Namen *Spiel.java* gespeichert. Quelltextdateien können mit jedem Texteditor bearbeitet werden. Es gibt sehr viele frei verfügbare oder kostengünstige Texteditoren. Einige, wie *Notepad* oder *Wordpad*, werden mit Windows ausgeliefert; aber wenn Sie den Editor für mehr als nur für einen schnellen Test benutzen wollen, dann werden Sie sich schnell etwas Besseres besorgen wollen. Seien Sie jedoch vorsichtig mit Textverarbeitungsprogrammen; diese speichern üblicherweise nicht im einfachen Textformat, und das Java-System kann die Dateien dann nicht lesen.

Die Quelltextdateien können dann in der Kommandozeile übersetzt werden, indem Sie den Compiler aufrufen, der mit dem JDK ausgeliefert wird. Er heißt **javac**. Um eine Quelltextdatei *Spiel.java* zu übersetzen, benutzen Sie das Kommando

```
javac Spiel.java
```

Dieses Kommando wird die Klasse **Spiel** sowie alle Klassen, die von dieser Klasse abhängen, übersetzen und dann eine Datei *Spiel.class* anlegen. Diese Datei enthält den Code, der von der virtuellen Maschine für Java ausgeführt werden kann. Um sie auszuführen, benutzen Sie das Kommando

```
java Spiel
```

Beachten Sie, dass bei diesem Aufruf die Endung *.class* nicht mit angegeben ist.

Lizenziert für Hochschule Luzern, 147.88.200.39 am 22.02.2023 um 10:57 Uhr

ANHANG

F

Benutzung des Debuggers

Der Debugger in BlueJ bietet eine Reihe von absichtlich vereinfachten, aber sehr nützlichen Funktionen zum Debuggen von Programmen und zum leichteren Verstehen des Laufzeitverhaltens von Programmen.

Das Debugger-Fenster wird hervorgeholt, wenn der Eintrag DEBUGGER ANZEIGEN aus dem Menü ANSICHT ausgewählt wird oder wenn über dem Aktivitätsbalken mit der rechten Maus geklickt wird und der Eintrag ZEIGE DEBUGGER aus dem Kontextmenü ausgewählt wird. Abbildung F.1 zeigt das Debugger-Fenster.

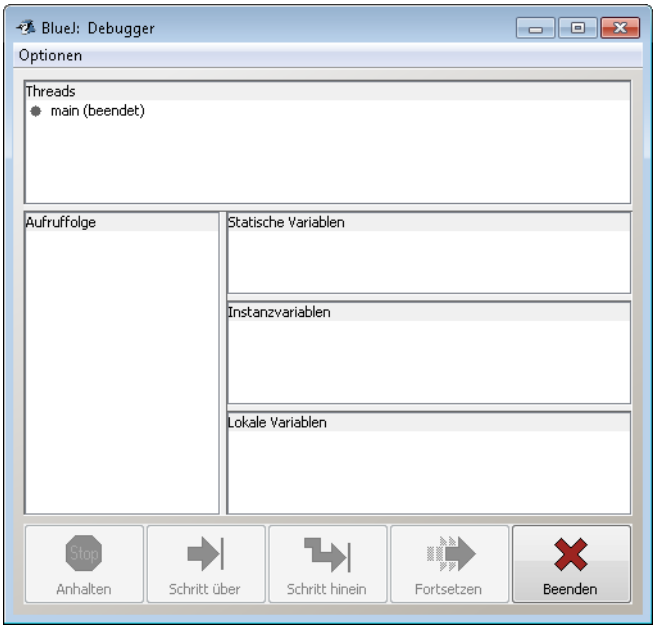


Abbildung F.1
Das Debugger-Fenster von BlueJ.

Das Debugger-Fenster enthält fünf Anzeigebereiche und fünf Kontrollknöpfe. Die Bereiche und die Knöpfe werden nur aktiviert, wenn ein Programm einen Haltepunkt erreicht oder aus einem anderen Grund anhält. Die folgenden Abschnitte beschreiben, wie Haltepunkte gesetzt werden, wie die Programmausführung kontrolliert wird und welchen Zweck die verschiedenen Anzeigebereiche haben.

Lizenziert für Hochschule Luzern, 147.88.200.39 am 22.02.2023 um 10:57 Uhr

F.1 Haltepunkte

Ein Haltepunkt ist ein Merker, der einer Quelltextzeile angeheftet wird (Abbildung F.2). Wenn ein Haltepunkt während der Programmausführung erreicht wird, werden die Debugger-Anzeigen und -Knöpfe aktiv, sodass Sie den Zustand des Programms untersuchen und die weitere Ausführung kontrollieren können.

Abbildung F.2
Ein Haltepunkt, der einer Quelltextzeile angeheftet ist.

```
36
37      /**
38       * Gib die nächste Nachricht (falls vorhanden) für den
39       * Benutzer dieses Clients auf der Konsole aus.
40       */
41      public void naechsteNachrichtAusgeben()
42      {
STOP        Nachricht nachricht = server.gibNaechsteNachrichtFuer(benutzer);
44          if(nachricht == null) {
45              System.out.println("Keine neue Nachricht.");
46          }
47          else {
48              nachricht.ausgeben();
49          }
50      }
51
```

Haltepunkte werden im Editorfenster gesetzt. Entweder klicken Sie mit der linken Maustaste in den Haltepunktbereich links vom Quelltext oder Sie platzieren den Cursor in der Zeile, in der der Haltepunkt gesetzt werden soll, und selektieren dann im Menü WERKZEUGE den Eintrag HALTEPUNKT SETZEN/ENTFERNEN. Auf die gleiche Weise können Haltepunkte auch wieder entfernt werden. Haltepunkte können nur in übersetzten Klassen gesetzt werden.

F.2 Die Kontrollknöpfe

Abbildung F.3 zeigt die Kontrollknöpfe, die an einem Haltepunkt aktiv sind.

Abbildung F.3
Aktive Kontrollknöpfe an einem Haltepunkt.

F.2.1 Anhalten

Der ANHALTEN-Knopf ist nur während einer Programmausführung aktiv und erlaubt es, bei Bedarf die Ausführung zu unterbrechen. Wenn die Ausführung hält, zeigt der Debugger den Zustand des Programms so an, also wäre dieses an einem Haltepunkt gestoppt worden.

F.2.2 Schritt über

Der SCHRITT ÜBER-Knopf setzt die Ausführung bei der aktuellen Anweisung fort. Nach Ausführung der Anweisung wird erneut gehalten. Wenn die Anweisung einen Methodenaufruf einschließt, dann wird dieser Aufruf komplett abgeschlossen, bevor die Ausführung angehalten wird (es sei denn, der Aufruf führt zu einem weiteren expliziten Haltepunkt).

Lizenziert für Hochschule Luzern, 147.88.200.39 am 22.02.2023 um 10:57 Uhr

F.2.3 Schritt hinein

Der SCHRITT HINEIN-Knopf setzt die Ausführung bei der aktuellen Anweisung fort. Wenn die Anweisung ein Methodenaufruf ist, dann führt die Ausführung in die Methode hinein und hält bei der ersten Anweisung innerhalb dieser Methode.

F.2.4 Fortsetzen

Der FORTSETZEN-Knopf setzt die Ausführung fort, bis entweder der nächste Haltepunkt erreicht wird oder die Ausführung mit dem ANHALTEN-Knopf gestoppt wird oder die Ausführung regulär endet.

F.2.5 Beenden

Der BEENDEN-Knopf bricht das aktuelle Programm sofort ab, sodass mit der Ausführung nicht mehr fortgefahren werden kann. Wenn Sie die Ausführung nur unterbrechen wollen, um den aktuellen Programmstatus zu untersuchen, sollten Sie den ANHALTEN-Knopf betätigen.

F.3 Anzeige der Variablen

Abbildung F.4 zeigt alle drei an einem Haltepunkt aktiven Anzeigebereiche für Variablen in einem Beispiel aus der Jäger-Beute-Simulation, die in Kapitel 12 diskutiert wird. Statische Variablen werden im oberen Bereich angezeigt, Instanzvariablen im mittleren und lokale Variablen im unteren Bereich.

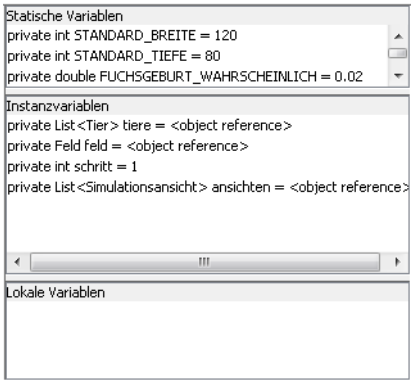


Abbildung F.4
Aktive Anzeigebereiche für Variablen.

Wenn ein Haltepunkt erreicht wird, wird die Ausführung an einer Anweisung eines beliebigen Objektes innerhalb des Programms angehalten. Der Bereich *Statische Variablen* zeigt die Werte der statischen Variablen, die in der Klasse dieses Objektes definiert sind. Der Bereich *Instanzvariablen* zeigt die Werte der Instanzvariablen des betreffenden Objektes. Beide Bereiche zeigen auch alle Variablen an, die von Superklassen geerbt wurden.

Lizenziert für Hochschule Luzern, 147.88.200.39 am 22.02.2023 um 10:57 Uhr

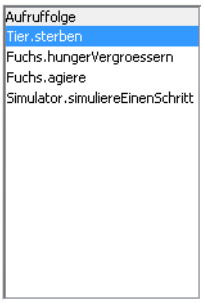
Der Bereich *Lokale Variablen* zeigt die Werte der lokalen Variablen und Parameter der gerade ausgeführten Methode bzw. des gerade ausgeführten Konstruktors. Lokale Variablen erscheinen in diesem Bereich erst dann, wenn sie initialisiert wurden, da sie erst zu diesem Zeitpunkt innerhalb der virtuellen Maschine sichtbar werden.

Jede Variable in diesen Bereichen, die auf ein Objekt verweist, kann mit einem Doppelklick weiter untersucht werden.

F.4 Die Anzeige der Aufruffolge

Abbildung F.5 zeigt den Fensterausschnitt für die Anzeige der Aufruffolge, hier mit einer Sequenztiefe von vier Methoden. Die Methoden erscheinen im Format `Klasse.methode` in der Sequenz, unabhängig davon, ob es sich um eine Klassen- oder Instanzmethode handelt. Konstruktoren erscheinen als `Klasse.<init>` in der Sequenz.

Abbildung F.5
Eine Aufruffolge.



Die Aufruffolge arbeitet nach dem Stapelprinzip, bei dem die Methode ganz oben diejenige bezeichnet, in der gerade der Kontrollfluss hält. Die Variablen-Anzeigen zeigen jeweils Details zu der Methode oder dem Konstruktor, der aktuell in der Aufruffolge hervorgehoben ist. Das Selektieren einer anderen Zeile aus der Folge bewirkt, dass die Inhalte der anderen Anzeige-Bereiche aktualisiert werden.

F.5 Die Thread-Anzeige

Die Thread-Anzeige geht über den Rahmen dieses Buches hinaus und wird hier nicht weiter behandelt.

Lizenziert für Hochschule Luzern, 147.88.200.39 am 22.02.2023 um 10:57 Uhr

ANHANG

G

Testwerkzeuge für Modultests mit JUnit

In diesem Anhang geben wir einen kurzen Überblick über die wichtigsten Eigenschaften der BlueJ-Unterstützung für Modultests im JUnit-Stil. Weitere Einzelheiten können im Test-Tutorial nachgelesen werden, das auf der BlueJ-Website verfügbar ist.

G.1 Aktivieren der Test-Funktionalität

Um die Test-Funktionalität von BlueJ zu aktivieren, müssen Sie sicherstellen, dass unter WERKZEUGE/EINSTELLUNGEN/DIVERSES die Box *Testwerkzeuge anzeigen* selektiert ist. Das Hauptfenster von BlueJ enthält dann einige weitere Knöpfe, die bei einem geöffneten Projekt aktiviert sind.

G.2 Eine Testklasse erzeugen

Sie erzeugen eine Testklasse, indem Sie einen Rechtsklick auf einer Klasse im Klassendiagramm ausführen und im Kontextmenü TESTKLASSE ERZEUGEN auswählen. Der Name der Testklasse wird automatisch festgelegt, indem Test als Suffix für den Namen der zugeordneten Klasse verwendet wird. Alternativ können Sie eine Testklasse auch über den Knopf *neue Klasse…* erzeugen, indem Sie als Klassentyp *Unit-Test* angeben. Dann können Sie den Namen der Klasse frei festlegen.

Testklassen werden mit `<<unit test>>` im Klassendiagramm annotiert und haben eine andere Farbe als gewöhnliche Klassen.

G.3 Eine Testmethode erzeugen

Testmethoden können interaktiv erzeugt werden. Eine Sequenz von Benutzerinteraktionen mit dem Klassendiagramm und der Objektleiste werden aufgezeichnet und als eine Folge von Java-Anweisungen und Deklarationen in einer Methode der Testklasse festgehalten. Sie starten eine Aufzeichnung, indem Sie ERZEUGE TESTMETHODE aus dem Kontextmenü einer Testklasse selektieren. Sie werden dann nach dem Namen der neuen Testmethode gefragt. Frühere JUnit-Versionen (bis Version 3) setzten voraus, dass dieser Name mit *test* begann. Dies ist heute nicht mehr der Fall. Das *Aufzeichnung*-Symbol links vom Klassendiagramm erscheint nun rot und die Knöpfe *Beenden* und *Abbrechen* werden verfügbar.

Lizenziert für Hochschule Luzern, 147.88.200.39 am 22.02.2023 um 10:57 Uhr